

QUALITÉ D'UNE INTERFACE DE CLASSE

Critères de qualité d'une interface

- Cohérente:
 - une classe décrit une seule abstraction.
 - Toutes les opérations doivent remplir un seul but.
- Opérations primitives:
 - opérations non décomposables.
- Complète:
 - toutes les opérations qui ont du sens sur l'abstraction devraient être supportées.



Critères de qualité d'une interface

- Pratique:
 - ... accommoder les usagers de la classe en ajoutant des opérations non primitives
- Consistante:
 - I.. Les opérations devraient être consistantes entre elles :
- sur la base de leur nom, arguments, valeur de retour et comportement.

Interface cohérente

- Une seule abstraction...
- Exemple : la classe string de la librairie standard.
 - Classe contenant un certain nombre de méthodes concernant les chaînes de caractères.
- Et si parmi ces méthodes se retrouvait la méthode `getCurrentDate()`
- ,...?

Opérations primitives

- Opérations non décomposables en d'autres opérations.
- Exemple : classe string
- offre l'opération insertUpper
 - consisterait en l'insertion d'une chaîne de caractères et en la mise en majuscule de celle-ci.
- La classe doit offrir ces services en deux opérations séparées.

Interface complète

- Toutes les opérations qui ont du sens pour la classe devraient être présentes.
- Exemple : classe string de la librairie standard
 - Si possibilité d'insérer une string à l'intérieur d'une string existante (insert)
 - Avoir aussi une opération d'effacement d'une sous chaîne (erase).

Interface pratique

- Commencer par une interface minimale.
- Ajouter tout doucement de nouvelles méthodes au besoin.
- Exemple : classe string
 - pourrait offrir un service de recherche de champ comme field (2).

«Hello cruel world»

Le champ no 2 serait ici «cruel»

Interface consistante

- Ne pas **mélanger** les façons de nommer les méthodes: asgCeci et asg_cela.
- Si vous avez une méthode d'accès nommée reqVitesse(),
 - ne pas appeler la méthode d'assignation asgVelocite()
-

Interface consistante

- Pour une fonction booléenne,
 - Toujours retourner true en cas de succès et false autrement.
-

Qualité d'une interface de classe

- La théorie du contrat
-

Écrire le contrat

- Lorsque vous engagez un sous-contractant
 - vous devez vous assurer que celui-ci exécute la tâche telle que requise.
- Possible uniquement si vos attentes sont inscrites dans un contrat en bonne et dû forme.

Un contrat protège les deux parties

- Protège le client(dans ce cas ci vous-mêmes)
 - Spécifie quelles sont ses obligations.
 - En retour, est assuré d'un certain résultat.
- Protège le fournisseur(sous-contractant)
- Spécifie le minimum requis (de la part du client):
 - le fournisseur ne peut être tenu responsable d'avoir manqué à des engagements ne figurant pas dans le contrat.

La spécification d'un contrat

- Spécification d'une interface
 - peut être vue comme un contrat entre un **client**(l'utilisateur du composant/classe) et un **fournisseur**(le composant/classe).
- Le contrat dit ce qui doit être rencontré par le client
 - pour pouvoir appeler une opération de l'interface :

La spécification d'un contrat

- **Précondition**
- Le contrat dit aussi ce qui doit être réalisé par le fournisseur
 - de fait, ce qui est garanti au client
 - pour rencontrer ce qui est prévu par l'opération
- **postcondition.**

Pour garantir la fiabilité

- Un logiciel peut être vu comme un ensemble d'échanges de services entre les composants.
- L'approche par contrat utilise la métaphore des contrats commerciaux
 - ce qui est exigé du client qui utilise le service;
 - ce qui en contre-partie est garanti par le service.

Pour garantir la fiabilité

- Les contrats permettent de spécifier de façon non ambiguë chacun des services.
- Doit être explicite dans les spécifications.
- Doit aussi être implanté à même le code source.

Les assertions : le contrat du logiciel

- Pour produire du logiciel correct et robuste :
 - rendre les obligations et les garanties explicites pour chaque appel de méthode.
- Mécanismes pour exprimer ces conditions: **assertions**.
- **Préconditions** et **postconditions**:
 - assertions s'appliquant à des méthodes en particulier,

Les assertions : le contrat du logiciel

- **Invariants** de classe :
 - assertions s'appliquant à toutes les méthodes d'une classe en tout temps.

Pour une spécification complète

- La seule signature d'une opération de l'interface d'un composant ne permet pas de tout savoir.
- La signature comprend:
 - le nom de l'opération
 - le type et l'ordre des paramètres entrants/sortants
 - le type de retour

Pour une spécification complète

- Toutefois, on ne sait pas:
 - dans quel contexte l'opération peut être appelée,
 - Quels résultats sont escomptés.
- Le contrat permet de clarifier cet aspect.

Pour une spécification complète

- Une spécification plus complète d'une opération offerte par un composant deviendrait:
 - ☐ le nom de l'opération
 - ☐ le nombre, le type et l'ordre des paramètres
 - ☐ Les préconditions
 - ☐ Les postconditions
 - ☐ le type de retour.

Invariant de classe

- Autre type d'assertion
 - représente l'ensemble des *conditions logiques* devant être respectées pour assurer un comportement correct d'une classe.
- Immédiatement après la création d'un objet de cette classe,
 - l'invariant doit être *respecté* et *préservé* tout au long de la vie de l'objet.

Invariant : exemple

- Classe *Date* :
- L'invariant pourrait être la validation de la cohérence entre les valeurs des attributs formant nécessairement une date valide.
-

Invariant : exemple

```
class Date
{
public:
    // ...
private:
    int jour;
    int mois;
    int annee;
};
```

Invariant

- Validation utile à l'utilisateur de la classe
- tout autant qu'au programmeur de celle-ci.
-

Validation logique:

- `jour >= 1`
- `mois >= 1 && mois <= 12`
- `annee != 0`
- `Si mois == 2 && jour == 29 ->`
 `estBisextile(annee)`
- `jour < nbJourDansMois étant donné`
 `{31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31}`

Préserver les invariants de classe

- Toutes les méthodes de la classe doivent préserver les invariants.
 - Au milieu d'une méthode, les invariants peuvent être violés mais doivent être rétablis à la fin de celle-ci.
- Lorsque la classe semble avoir des états transitoires invalides,
 - on est tenté d'affaiblir les invariants.
 - d'abord envisager la possibilité d'empêcher l'existence de ces états transitoires.

Préserver les invariants de classe

- Les invariants doivent donc être respectés (et testés) :
 - à la fin de tous les constructeurs.
 - à la fin de toutes les méthodes ***non constantes***,
 - i.e. les méthodes qui *modifient* l'objet.

Compléter le contrat

- Les *invariants* de classe ne suffisent pas pour spécifier ce qui est permis et interdit en tout temps.
- Exemple :
- l'*invariant* d'une Pile ne dit rien sur ce qui doit se passer lors de l'ajout d'un élément sur une pile *pleine*.

- *Précondition* de la méthode `push(...)`.
On doit tester la condition! `estPleine()`.

Compléter le contrat

- ... après avoir ajouter un élément sur la pile, nous savons que la condition! `estVide()` doit être vrai.
- ***Postcondition* de la méthode `push(...)`.**
- Méthodes de classe vues comme des agents remplissants un ***contrat***.
Un contrat implique des responsabilités de parts et d'autres.

Préconditions et postconditions

- Les ***préconditions*** :
- conditions devant être remplies ***avant d'exécuter une méthode.***
- Si le contrat n'est pas rempli, la méthode n'a pas à s'exécuter. [Erreur de programmation]
-

Préconditions et postconditions

- Les *postconditions*:
- conditions devant être remplies *après* l'exécution d'une méthode.
- Si le contrat n'est pas rempli, la méthode ne doit pas retourner. [Erreur de programmation]

Implanter la théorie du contrat en C++

- Le langage C++ n'offre à la base aucun support pour implanter ce genre de concept.
- Implantation généralement retenue pour mettre sur pied la mécanique basée sur les **assertions**

Implanter la théorie du contrat en C++

- *Une précondition* ou une *postcondition* prend la forme d'une expression booléenne.

Le modèle le plus simple des assertions :
Version disponible dans le fichier `assert.h` de la librairie standard:

```
assert(«condition logique»);
```

